

Web App & DevOps Training Plan

From Git fundamentals to a deployed
and monitored object-detection web application

Dr. Phùng Kieu Hà
SEEE, HUST

Course Information

- **Duration:** 7 weeks
- **Project:** YOLO image/video object-detection web application
- **Backend:** Python + FastAPI + Ultralytics YOLO
- **Frontend:** React + Vite (simple upload and result viewer)
- **Infrastructure:** Docker, Docker Compose, GitHub Actions, cloud deployment
- **Learning approach:** One continuous project; each week adds a production capability

Overall Learning Outcomes

By the end of this training, learners will be able to:

- Use Git and GitHub confidently in a team workflow.
- Build a backend API that detects objects from uploaded images and short videos.
- Build a frontend that uploads media, calls the backend, and displays detection results.
- Containerize the frontend and backend and run them together with Docker Compose.
- Create a CI/CD workflow that tests, builds, and deploys the application.
- Deploy the application to a cloud platform and manage configuration securely.
- Use logs, health checks, monitoring, and a repeatable debugging process.
- Complete and present a production-style final project.

Training Structure

The course is organized into 7 weeks, each adding one production capability to a single continuous project, following the principle that every week builds on the work of the previous one.

- Week 1 Git, GitHub, and Team Workflow
- Week 2 Build a Simple YOLO Web Application
- Week 3 Docker and Containerization
- Week 4 CI/CD with GitHub Actions
- Week 5 Deploying to the Cloud
- Week 6 Logging, Monitoring, and Debugging
- Week 7 Final Project and Demo

Week 1: Git, GitHub, and Team Workflow

Objective: Learn the Git workflow that will be used throughout the training project.

Documents

- [Git documentation](#)
- [GitHub flow](#)
- [Resolving merge conflicts](#)

Requirements

- LINUX!
- Register or prepare a GitHub account and configure Git locally.
- Understand repository, commit, branch, remote, pull request, merge, rebase, tag, and release.
- Practice clone, status, add, commit, log, diff, branch, switch, pull, push, fetch, merge, and rebase.
- Use a simple team workflow: main → feature branch → pull request → review → merge.
- Practice resolving one intentional merge conflict.
- Learn useful repository files: README.md, .gitignore, LICENSE, and CONTRIBUTING.md.
- Use clear branch names such as `feature/backend-image-detection` and `fix/video-upload-error`.
- Use meaningful commit messages rather than large or unclear commits.

Deliverables

- Personal practice repository with at least three feature branches and pull requests.
- One documented merge-conflict exercise.
- Initial team repository for the YOLO application.
- README containing project goal, team members, branch rules, and local prerequisites.

Acceptance criteria

- Every members can create a branch, commit changes, push it, open a pull request, and resolve a basic conflict.
- The main branch is protected from direct feature development.
- The repository history is understandable and contains no secrets or generated files.

Week 2: Build a Simple YOLO Web Application

Objective: Create one backend for image/video detection and one frontend for upload and result viewing.

Documents

- [Ultralytics YOLO documentation](#)
- [FastAPI documentation](#)
- [FastAPI file uploads](#)
- [React documentation](#)
- [Vite documentation](#)

Requirements

- Understand the request flow: browser → frontend → backend API → YOLO model → response → frontend display.
- Create a backend with FastAPI and load a small pretrained YOLO model once at startup.
- Implement `POST /api/detect/image` for JPG, JPEG, or PNG uploads.
- Return useful metadata: detected class, confidence score, bounding-box coordinates, count, and processing time.
- Return an annotated image or a URL from which the frontend can retrieve it.
- Implement `POST /api/detect/video` for short MP4 clips. Process frames, create an annotated output video, and return summary counts.

- Add `GET /health` for later Docker and cloud health checks.
- Validate file type and size and return clear 4xx/5xx errors.
- Create a React frontend with image/video upload, preview, confidence threshold, loading state, error state, and result display.
- Enable CORS only for the required frontend origin.
- Keep the first version simple: no database, login, custom model training, or long-running video queue.

Deliverables

- `backend/` and `frontend/` folders in one repository.
- Working image-detection flow from browser to backend and back.
- Working short-video detection flow.
- OpenAPI documentation available through FastAPI.
- Updated README with local setup, API endpoints, supported formats, limitations, and screenshots.

Acceptance criteria

- An uploaded image is annotated and displayed in the browser.
- A short uploaded video is processed and the result can be viewed or downloaded.
- Invalid files produce clear user-facing errors without crashing the server.
- At least three backend tests cover health check, valid image upload, and invalid file handling.

Week 3: Docker and Containerization

Objective: Run the complete application consistently without depending on each member's local Python or Node.js setup.

Documents

- [Docker get started](#)
- [Dockerfile reference](#)
- [Docker Compose documentation](#)
- [Docker build best practices](#)

Requirements

- Understand image, container, registry, layer, volume, network, port, environment variable, and health check.
- Write a backend Dockerfile with dependency caching and a non-root runtime user.
- Write a multi-stage frontend Dockerfile that builds the React app and serves static files.
- Create `compose.yaml` to run frontend and backend on one Docker network.
- Use environment variables for API base URL, model name, upload limit, and confidence defaults.
- Add `.dockerignore` files to reduce build context.
- Persist generated results only when required; otherwise clean temporary files after processing.
- Add container health checks and restart behavior.
- Compare development execution with containerized execution.

Deliverables

- Backend Dockerfile, frontend Dockerfile, and `compose.yaml`.
- `.dockerignore` and `.env.example` files.
- One-command startup using `docker compose up --build`.
- Documented build, run, stop, logs, and cleanup commands.

Acceptance criteria

- A new member can run the full application with Docker by following the README.
- The frontend reaches the backend by service name inside Compose rather than hard-coded local addresses.
- Containers start cleanly, expose only required ports, and report healthy status.

Week 4: CI/CD with GitHub Actions

Objective: Automatically validate every change and produce deployable container images.

Documents

- [GitHub Actions documentation](#)
- [Building and testing Python](#)
- [Publishing Docker images](#)
- [Docker build-push action](#)

Requirements

- Understand continuous integration, continuous delivery, jobs, steps, runners, triggers, artifacts, cache, secrets, and environments.
- Create a pull-request workflow that installs dependencies, checks formatting/linting, and runs backend and frontend tests.
- Build both Docker images in CI to catch Dockerfile errors.
- Create a main-branch workflow that tags and pushes images to GitHub Container Registry or another approved registry.
- Use dependency and Docker layer caching where appropriate.
- Use GitHub secrets for registry or deployment credentials; never commit credentials.
- Add status checks as pull-request merge requirements.
- Optionally generate a release image tag from a Git tag such as `v1.0.0`.

Deliverables

- `.github/workflows/ci.yml`.
- `.github/workflows/build-and-publish.yml` or one clearly structured combined workflow.
- Visible successful workflow runs and published container images.
- CI badge in README.

Acceptance criteria

- A failing test prevents the pull request from passing.
- A successful merge to main builds versioned frontend and backend images.
- Workflow files contain no plaintext credentials.
- The team can explain the path from push to test, image build, registry, and deployment.

Week 5: Deploying to the Cloud

Objective: Deploy the containerized application to a publicly accessible cloud environment.

Documents

- [Render Docker deployment](#)
- [Render environment variables and secrets](#)
- [Render health checks](#)
- [GitHub environments](#)

Requirements

- Choose one supported cloud approach, preferably Render for the beginner version.
- Deploy the backend as a containerized web service and the frontend as a static site or containerized service.
- Configure the frontend with the public backend URL.
- Configure secrets and environment variables in the cloud dashboard or GitHub environment.
- Configure `/health` as the service health-check path.
- Handle cloud limits: CPU-only inference, memory usage, cold starts, request timeout, upload size, and temporary storage.
- Use a small YOLO model and restrict video duration/resolution for a reliable beginner deployment.
- Configure deployment from main only after CI succeeds.
- Use HTTPS URLs and verify CORS settings for the deployed frontend domain.
- Document rollback or redeployment steps.

Deliverables

- Live frontend URL and live backend health URL.
- Successful end-to-end image detection in the cloud.
- Successful short-video test within documented platform limits.
- Deployment guide including environment variables, secrets, health checks, limitations, and rollback steps.

Acceptance criteria

- The live application can be opened by another person without local setup.
- No secret is exposed in source code, browser code, logs, or screenshots.
- A failed deployment does not replace the last healthy version when the platform supports safe rollout behavior.
- The README clearly states resource and media-size limitations.

Week 6: Logging, Monitoring, and Debugging

Objective: Make failures observable and teach members to diagnose problems systematically.

Documents

- [Python logging documentation](#)
- [FastAPI handling errors](#)
- [Render logging](#)
- [Docker logs](#)

Requirements

- Use structured backend logs with timestamp, level, request ID, route, status, latency, media type, media size, model name, and detection count.
- Do not log uploaded media, authentication secrets, tokens, or sensitive file paths.
- Add clear exception handling around file validation, model loading, inference, video encoding, and temporary-file cleanup.
- (Optional) Expose health information through `/health` and optionally `/ready` for model readiness.
- Use cloud and Docker logs to investigate errors.
- Monitor basic signals: request count, error rate, response time, restart count, CPU, and memory.
- Add frontend error messages that include a safe request ID for log correlation.
- Practice debugging scenarios: incorrect backend URL, CORS failure, unsupported codec, oversized file, out-of-memory restart, missing model file, and failed CI build.
- Write a troubleshooting decision tree: reproduce → collect evidence → isolate layer → fix → test → document.
- Review Git, Docker, CI/CD, and deployment concepts through one controlled incident exercise.

Deliverables

- Consistent logging configuration in the backend.
- Health/readiness endpoints and cloud health-check configuration.
- Troubleshooting and debugging guide.
- Incident exercise report containing symptoms, evidence, root cause, fix, and prevention.

Acceptance criteria

- A member can trace one failed browser request through frontend output, backend logs, and deployment logs.
- Logs are useful but do not leak secrets or uploaded content.
- Common failures return actionable errors instead of generic crashes.
- The service recovers cleanly after a controlled restart.

Week 7: Final Project and Demo

Objective: Integrate all learned skills into a complete, reviewable, and deployed application.

Documents

- [GitHub repository documentation](#)
- [Writing a README](#)

Requirements

- Work in teams of two to four using branches, pull requests, reviews, issues, and a small project board.
- Define a narrow detection use case, such as people/vehicle counting, classroom-object detection, or safety-equipment detection when an appropriate model is available.
- Complete the image and short-video user flows.
- Add at least one useful feature: class filtering, object counts, confidence control, result history for the current session, or downloadable detection summary.

- Add backend tests and at least one frontend test or automated UI check.
- Run the system through Docker Compose.
- Use CI to test and build container images.
- Deploy the final version to the cloud.
- Provide logging, health checks, and a documented debugging scenario.
- Prepare a short presentation and live demonstration.

Deliverables

- Complete GitHub repository with clean history and reviewed pull requests.
- Live application URL and container images.
- Architecture diagram showing browser, frontend, backend, YOLO, registry, CI/CD, and cloud services.
- README with project overview, architecture, local setup, Docker setup, API contract, CI/CD flow, deployment, monitoring, troubleshooting, limitations, and future work.
- Final presentation and live demo.
- Individual reflection describing contribution and lessons learned.

Acceptance criteria

- The application detects objects from both images and short videos.
- The frontend and backend are independently structured and communicate through a documented API.
- Tests, Docker builds, and CI checks pass.
- The deployed version is reachable and health checks pass.
- The team can explain one limitation and one realistic improvement rather than claiming perfect accuracy.

Final project score distribution

-	Weight	Evidence
Git and teamwork	15%	Branches, pull requests, reviews, commit quality, issue tracking
Backend and YOLO integration	20%	Correct API behavior, validation, model reuse, image/video processing
Frontend experience	15%	Upload, loading, errors, result display, usability
Docker and reproducibility	15%	Dockerfiles, Compose, configuration, one-command startup
CI/CD and cloud deployment	15%	Automated tests/builds, image publishing, working deployment
Logging, monitoring, debugging	10%	Useful logs, health checks, incident diagnosis
Documentation and presentation	10%	README, architecture, limitations, demo quality

Scope:

- Use a pretrained nano/small YOLO model; custom training is optional and outside the core plan.
- Limit video uploads by duration, resolution, and file size to avoid request timeouts and memory problems.

- Do not add authentication, a database, message queues, Kubernetes, or GPU orchestration unless the core project is already stable.
- Prefer a reliable image workflow over an ambitious but unstable real-time streaming system.
- Treat model predictions as uncertain outputs and display confidence scores and limitations clearly.